



Stablecoin Circuit Breaker

Detect stablecoin stress. Trigger the pre-approved response.
Prove every step on Solana.

Event: Saturday 26 September 2026 · Dogpatch Labs, Dublin Version: 2.0 (25 Sept 2026)

At a glance

Item	Summary
What it is	A web app that watches stablecoins, rates stress green / amber / red, proposes a pre-approved treasury action, gets a named manager's wallet sign-off, and records every step on Solana devnet.
Problem	Detecting a depeg is already solved by many tools. What firms lack is an agreed response in the moment and tamper-proof evidence afterwards.
Users	Treasury teams, fintechs and payment firms holding stablecoins, and their risk and compliance staff.
Solana's role	Notary (memo records), sign-off (Phantom signature) and executor (test-token transfer on devnet).
Demo	A scenario player with a Play button replays four simulated events, including a stylised March 2023 USDC depeg.
Build	One web page built with Claude Code, in four tiers. Each tier works alone, so there is always a demo.
Honest limits	Flags stress, never predicts depegs. Simulated data and devnet test tokens only. Thresholds are assumptions.

How to use this brief. Sections 1–4 give you the story for the pitch deck. Sections 5–9 are the product spec for Claude Code. Section 10 has copy-paste build prompts. Section 11 defines the **proof of project**: exactly what judges must see to believe it works. Section 13 is a slide-by-slide plan for the PPTX.

1 The project in one paragraph

Stablecoin Circuit Breaker detects stress in the stablecoins a firm holds, triggers the firm's pre-approved response, and writes a tamper-proof record of every step on Solana. Existing tools stop at the warning. We add the two things that come after it: **a decision made by rules agreed in advance**, and **proof that those rules were followed**.

One-line pitch: “When USDC lost 13% of its value in March 2023, everyone could see the price. What most firms lacked was an agreed plan, and proof of what they did. Circuit Breaker gives them both.”

It is an early-warning and response tool. It does not predict depegs, trade real money, or give investment advice.

2 Background in plain English

A **stablecoin** is a digital token designed to stay worth \$1, like a casino chip you can always swap back for cash. Firms use stablecoins as digital cash: to pay, to move money across borders, and to hold funds. The biggest two, USDC and USDT, say every token is backed by cash and short-term government debt.

The **peg** is the \$1 target. A **depeg** is when the market price slides away from \$1 and stays away, because people doubt they can cash out at full value. Tiny, brief wobbles are normal. Large or lasting moves signal stress.

Event	What happened	Lesson for the pitch
TerraUSD, May 2022	A stablecoin backed by code rather than cash lost its peg and collapsed to almost zero.	Some depegs never recover.
USDC, March 2023	Fell to roughly 87 cents over a weekend after Silicon Valley Bank, which held part of its reserves, failed. It recovered within days.	Even the “safe” coins can break, and markets never close.

In the EU, the **MiCA** regulation sets rules for stablecoin issuers. That raises the pressure on firms that hold stablecoins to show how they manage this risk.

3 Problem, users and goal

Problem. Seeing a depeg is no longer the hard part; price trackers and alert bots already do that. The hard part is what comes next. Under stress, a treasury team must decide in minutes, often out of hours, with no agreed plan. Afterwards, it must prove to auditors and regulators what it saw, when, and what it did, and a spreadsheet or email trail can be edited after the fact.

Target users. Treasury teams, fintechs and payment firms that hold or accept stablecoins, plus the risk and compliance staff who oversee them.

Goal. Answer three questions:

1. Is any stablecoin we hold under stress right now?
2. What does our agreed policy say we should do about it?
3. Can we prove, to anyone, exactly what we saw and did, and when?

4 What already exists, and the gap we fill

The original idea (a live price monitor with green / amber / red alerts) is already well served. Judges who know the space will recognise every feature of it. We therefore do not compete on detection. We compete on what happens after the alert.

Existing example	What it already does	Overlap with the original idea	What it doesn't do
kcolbchain / depeg-monitor (open-source, GitHub)	Watches USDT, USDC and DAI across exchanges; alerts to Discord, Slack or webhook.	Default warning 0.5% and critical 1%, almost identical to our thresholds.	Alerts only.
CoinGecko depeg-detection guide	Step-by-step tutorial for building real-time depeg alerts on its API.	The core monitor is effectively a published recipe.	A tutorial, not a product.
Webacy Stablecoin Monitor (commercial)	Risk tiers, depeg alerts, mint/burn “bank-run” monitoring, forward-looking stress signals.	Our supply signal and rating tiers, in a more advanced form.	No firm-specific policy or approval trail.
S&P; Global Stablecoin Stability Assessments	Rates stablecoins 1 (very strong) to 5 (weak); also published on-chain via Chainlink.	Risk rating, and even “ratings on-chain”.	Slow structural view, not minute-by-minute.
DefiLlama Stablecoins pages	Free dashboard of % off peg, supply changes and market cap per chain.	Our dashboard; a community page there is even named “The Stablecoin De-Peg Risk Monitor”.	No alerts, policy or audit log.
Apify depeg-monitor bots	Scheduled peg and supply checks with Telegram alerts.	Scheduled alerts and supply tracking.	No approval step or verifiable record.

Our gap in one sentence: detection is crowded, so Circuit Breaker sits one step later, on **response and evidence**. It turns an alert into a pre-approved action, signed off by a named person, with every step provable on Solana. Say this in the first 20 seconds of the pitch.

5 The solution: Detect, Respond, Prove

Think of a building's fire system: an alarm (Detect), sprinklers that follow a plan (Respond), and a logbook nobody can rewrite (Prove).

Layer	What it does	What the user sees
1. Detect	Rates each coin green / amber / red from price, volatility and supply signals. A colour must persist for a set time before it counts, so a one-second blip does not trigger action.	Rating cards with a reason, e.g. “price 1.4% below \$1”; price chart with amber and red bands.
2. Respond	Checks the firm's pre-written policy. When a rule fires, it proposes an action. A named manager approves or rejects it with a Phantom wallet signature. Approved moves transfer test tokens on devnet.	Policy panel, pending-approval card with countdown, Approve / Reject buttons.
3. Prove	Writes every alert, approval, rejection and action to Solana as a memo, giving each a public, uneditable timestamp and the signer's address.	Auditor view: a plain-language timeline with an Explorer link per event.

End-to-end flow (the demo moment)

1. The scenario plays; USDC drifts below \$1 and its card turns **amber**. The amber alert is logged to Solana.
2. The drift worsens and holds **red** for 5 simulated minutes. The policy rule fires.
3. The app proposes: “Move 50% of USDC holdings to the safe wallet (per policy rule R2).”
4. The manager clicks Approve and signs in Phantom. Test tokens move from the treasury wallet to the safe wallet.
5. The Auditor view shows: flagged → approved by [wallet] → moved, each with an Explorer link that opens a real devnet transaction.

Default treasury policy (editable in the app)

Rule	Condition	Action
R1	Any coin amber for 3 minutes	Notify the risk manager. Log alert. No funds move.
R2	Any coin red for 5 minutes	Propose moving 50% of that coin's holdings to the safe wallet. Needs manager approval.
R3	Proposal not approved within 2 minutes	Log “escalated, no action taken”. Nothing moves without a signature.

6 Architecture and components

Keep it to one web page for Tier 1, so it runs by double-clicking and cannot break on stage. For Tiers 2–3, Claude Code may turn it into a small Vite project (`npm run dev`) if the Solana libraries are easier to load that way.

Component	Job	Suggested tech
Scenario engine	Generates and replays mock minute-by-minute price and supply data; Play, Pause, Reset, speed.	JavaScript (data generated in code, no files)
Risk engine	Computes deviation, volatility and supply signals; worst-of rating; time-in-colour rule.	Plain JavaScript functions
Policy engine	Checks rules R1–R3; creates proposals; tracks approvals and timeouts.	Plain JavaScript
Dashboard	Rating cards, price-vs-\$1 chart with bands, policy panel, alert log, method panel.	HTML/CSS, Chart.js
Solana layer	Connects Phantom, sends memo transactions, sends test-token transfers, builds Explorer links.	@solana/web3.js, @solana/spl-token, Phantom, Memo program
Auditor view	Timeline of all events with on-chain links; export as CSV.	HTML table
Live mode (stretch)	Replaces simulated prices with live Pyth feeds.	Pyth Hermes API

7 Simulated scenario player

The demo runs on four built-in scenarios instead of hoping the live market misbehaves on Saturday. Each plays sped up (one simulated hour in about 60 seconds at 60×) under a clear **SIMULATION** banner.

Scenario	Price path	Expected result	Why it's in the demo
A. Calm day	\$1 ± 0.02% random noise for 2 hours	All green, no alerts	Shows it doesn't cry wolf
B. Flash wick	Calm, then -1.5% for 40 seconds, then back	Red flicker on screen, no alert logged, no action	Proves the time-in-colour rule filters noise
C. March 2023 replay (stylised)	Friday calm → Saturday slide to ~\$0.87 → Sunday stabilises ~\$0.90-0.95 → Monday recovers to ~\$0.99	Amber alert → red alert → R2 proposal → approval → tokens move → recovery back to green	The main demo moment; based on a real event
D. Collapse (Terra-style)	Supply falls 5% in an hour first, then price slides and accelerates to ~\$0.30, never recovers	Supply signal turns red before price; policy fires early	Shows why several signals beat price alone

Data rules for Claude Code

- Generate all data in JavaScript at page load using a fixed random seed, so every run looks the same on stage.
- One reading per simulated minute: {t, price, supply} for each coin (USDC, USDT, and optionally PYUSD).
- Only the scenario's target coin moves; the others stay calm, which makes the contrast visible.
- Label scenario C on screen as “stylised shape of the March 2023 event, not exact historical data”.

8 Risk rating rules

Overall rating = the **worst** of the three signal ratings. All thresholds are starting assumptions, not industry standards, and appear that way in the on-screen method panel.

Signal	How it is calculated	Green	Amber	Red
Price deviation	$ price - 1 \div 1 \times 100$	< 0.3%	0.3% - 1%	> 1%
Short-term volatility	Std. deviation of the last 15 readings, vs the coin's calm baseline	< 2× baseline	2× - 4×	> 4×
Supply change	% change in supply over the last 60 readings	< 1%	1% - 3%	> 3%

- **Time-in-colour rule:** the lights change instantly on screen, but the policy only acts once amber has held for 3 simulated minutes or red for 5.
- **Why worst-of:** if any warning light is red, the coin is red. It is conservative and takes one sentence to explain.
- **Calibration (next step):** tune thresholds against past events. Existing tools commonly warn at 0.5-1%, so ours are deliberately slightly more cautious.

9 What Solana does

Solana is essential here: it is both the executor and the notary. Take it out and the Respond and Prove layers stop working.

Role	How	Tools
Notary	Each alert, approval, rejection and action is written as a memo transaction on devnet: a public, uneditable timestamp.	Memo program, @solana/web3.js
Sign-off	The manager approves with their wallet signature, so the record shows <i>who</i> approved.	Phantom (devnet mode)
Executor	Approved moves transfer test tokens from the treasury wallet to the safe wallet, in the same transaction as the memo.	SPL test token (spl-token CLI), @solana/spl-token
Data (stretch)	Live stablecoin prices; token supply via RPC.	Pyth, getTokenSupply

Memo format (so records are readable and consistent)

```
CB|v1|<EVENT>|<COIN>|<RATING>|dev=<deviation%>|rule=<rule id>|sim=<simulated time>|scn=<scenario>
```

Examples:

```
CB|v1|ALERT|USDC|RED|dev=-2.31|rule=R2|sim=2023-03-11T02:14|scn=C
```

```
CB|v1|APPROVED|USDC|RED|move=50%|rule=R2|sim=2023-03-11T02:20|scn=C
```

```
CB|v1|MOVED|USDC|RED|amount=50000|to=SAFE|sim=2023-03-11T02:20|scn=C
```

No custom smart contract. Writing one in Rust is too slow for one day. Enforcing the policy inside an on-chain program is the natural next step to mention to judges.

10 Build guide for Claude Code

Build in tiers. Each tier is a working demo on its own. Paste one prompt at a time, test, then move on. Do not start Tier 3 before Tier 1 is solid.

Tier	What's included	Done when (acceptance test)
1. Must have	Scenario player, rating cards, chart with bands, time-in-colour rule, alert log, method panel. No blockchain.	All four scenarios play; B logs nothing; C logs amber then red.
2. Solid	Phantom connect; memo transaction per alert; Auditor view with Explorer links.	Clicking an alert's link opens a real devnet transaction showing the memo text.
3. Differentiator	Policy panel (R1-R3), approval card, Phantom-signed token transfer + memo.	One full run: flagged → approved → tokens moved → all three on Explorer; wallet balances change.
4. Stretch	Live Pyth prices toggle; real March 2023 history; CSV export.	Live cards show real prices, all green.

Friday setup (before any coding)

```
# 1. Install the Solana CLI - follow solana.com/docs/intro/installation
# (on Windows, install WSL first and run everything inside it)
solana --version

# 2. Point at devnet and create two wallets
solana config set --url devnet
solana-keygen new -o treasury.json
solana-keygen new -o safe.json

# 3. Free devnet SOL (if airdrop is rate-limited, use faucet.solana.com)
solana airdrop 2 $(solana-keygen pubkey treasury.json)

# 4. Create a test "stablecoin" and give the treasury 10,000 of it
spl-token create-token --decimals 6 --fee-payer treasury.json
spl-token create-account <TOKEN_MINT> --owner treasury.json --fee-payer treasury.json
spl-token mint <TOKEN_MINT> 10000 --fee-payer treasury.json
spl-token create-account <TOKEN_MINT> --owner safe.json --fee-payer treasury.json

# 5. Import treasury.json into Phantom and switch Phantom to Devnet
```

Write down the token mint address and both wallet addresses. Claude Code will need them for Tier 3.

Prompt 1 — Tier 1: the simulation

Build a single self-contained HTML file called circuit-breaker.html for a hackathon demo called "Stablecoin Circuit Breaker". No external data files. Load Chart.js from a CDN. Keep code simple and well commented.

DATA: In JavaScript, generate mock minute-by-minute readings {t, price, supply} for coins USDC and USDT using a fixed random seed. Four scenarios:

- A "Calm day": \$1 +/- 0.02% noise, 120 minutes.
 - B "Flash wick": calm, then USDC -1.5% for under 1 minute, then back.
 - C "March 2023 USDC replay (stylised)": calm Friday, Saturday slide to ~0.87, Sunday ~0.90-0.95, Monday recovery to ~0.99. Label it "stylised".
 - D "Collapse": USDC supply falls 5% in an hour, then price slides and accelerates to ~0.30 and never recovers.
- Only the target coin moves; the other stays calm.

RISK ENGINE: per coin compute (1) price deviation % (green <0.3, amber 0.3-1, red >1), (2) volatility = std dev of last 15 readings vs calm baseline (green <2x, amber 2-4x, red >4x), (3) supply change % over last 60 readings (green <1, amber 1-3, red >3). Overall = worst signal. Lights change instantly, but an ALERT is only logged when amber holds for 3 simulated minutes or red for 5.

UI: dark header with a "SIMULATION" banner. Controls: scenario dropdown, Play, Pause, Reset, speed 1x/10x/60x. Rating cards per coin with colour and reason text (e.g. "price 1.4% below \$1"). Line chart of price vs \$1 with shaded amber and red bands. Alert log with simulated timestamps. A method panel listing all thresholds labelled "starting assumptions". Design: clean, professional, readable from the back of a room.

Prompt 2 — Tier 2: log alerts on Solana

Extend circuit-breaker.html. Add a "Connect Phantom" button (devnet). When an alert is logged, send a devnet transaction using the Memo program (MemoSq4gqABAXKb96qnH8TysNcWxMyWCqXgDlGmfcHr) with this text format: CB|v1|ALERT|<COIN>|<RATING>|dev=<dev%>|rule=<rule>|sim=<time>|scn=<scenario> Phantom signs it. Show status (pending / confirmed / failed) in the alert log and a link: <https://explorer.solana.com/tx/<signature>?cluster=devnet> Add an "Auditor view" tab listing every event with time, coin, rating, signer address and link. If sending fails, keep the alert in the log marked "not on-chain" so the demo never breaks. Use @solana/web3.js. If loading it from a CDN is unreliable, convert the project to a small Vite app and tell me how to run it.

Prompt 3 — Tier 3: policy, approval and token move

Add a Policy panel with editable rules:

- R1: coin amber 3 min -> notify + log, no action.
- R2: coin red 5 min -> propose moving 50% of that coin's holdings to SAFE.
- R3: proposal not approved within 2 min -> log "escalated, no action".

When R2 fires, pause the simulation and show an approval card with the proposal, the rule, and Approve / Reject buttons plus a countdown. On Approve: build ONE devnet transaction containing (a) an SPL token transfer of the proposed amount of TEST TOKEN <MINT> from TREASURY <ADDRESS> to SAFE <ADDRESS> and (b) a memo "CB|v1|MOVED|...". Phantom signs it. On Reject: send a memo "CB|v1|REJECTED|...". Show both wallet balances on screen, refreshed after each move. Log every step in the Auditor view with Explorer links. Use @solana/spl-token for the transfer.

Test checklist before Saturday afternoon

- ☐ Scenario A runs for its full length with no alerts.
- ☐ Scenario B shows a red flicker but logs nothing.
- ☐ Scenario C logs amber, then red, then triggers R2.
- ☐ Each Explorer link opens and shows the memo text.
- ☐ Balances change after approval; a rejection moves nothing.

- ☐ The simulation still runs with Phantom disconnected (on-chain steps fall back gracefully). Chart.js loads from a CDN, so keep a phone hotspot as internet backup.

11 Proof of project

This is what turns “we built a thing” into “judges believe it works”. Every item below must be demonstrable on the day, and captured in the submission in case the live demo fails.

#	Proof point	How judges see it	Evidence to save
P1	The rating logic works	Scenario C: green → amber → red, with reasons on the cards	Screenshot of red card + chart
P2	It doesn't cry wolf	Scenario B: red flicker, but nothing logged or proposed	10-second screen clip
P3	Alerts are on-chain	Click an alert link; Solana Explorer shows the memo text	3+ devnet transaction links in README
P4	A named person approved	Explorer shows the approval signed by the manager's wallet address	Transaction link + wallet address
P5	The response executed	Treasury balance drops and safe balance rises by the proposed amount	Before/after balance screenshot
P6	The record is auditable	Auditor view shows the full timeline with links; CSV export	Exported CSV in the repo
P7	Honest method	Method panel shows thresholds labelled as assumptions; SIMULATION banner	Screenshot

Submission package (by 16:30)

- ☐ Public GitHub repo with the app and a README: problem, solution, how to run, tiers completed, limits.
- ☐ README section “Proof on Solana” listing the devnet transaction links and wallet addresses.
- ☐ A 60–90 second screen recording of scenario C end to end (backup if the live demo fails).
- ☐ 4–6 screenshots: dashboard, approval card, Explorer memo, Auditor view, method panel.
- ☐ The pitch deck (see section 13).

12 Timeline

Friday 25 September — prep

- ☐ Solana CLI installed, devnet wallets created and funded, test token minted (section 10).
- ☐ Phantom installed, treasury wallet imported, set to Devnet.
- ☐ Run Prompt 1 in Claude Code and get Tier 1 working if time allows.
- ☐ Send one memo transaction and confirm it on Explorer.
- ☐ Draft pitch deck slides 1–4 (problem, gap, solution).

Saturday 26 September — event day

Time	Milestone	Done when
10:00–11:00	Arrive, set up, confirm plan	Repo created; Tier 1 file opens
11:00–12:30	Tier 1 finished and tested	Test checklist items 1–3 pass
12:30–13:30	Tier 2: memo alerts + Auditor view	Proof point P3 works

Time	Milestone	Done when
13:30-15:00	Tier 3: policy, approval, token move	Proof points P4 and P5 work
15:00-15:30	Hard cut-off: if Tier 3 is shaky, stop and polish	Demo runs cleanly start to finish
15:30-16:30	README, screenshots, recording, submit	Submission sent before 16:30
16:30-17:00	Rehearse pitch + demo twice	Under 2 minutes, no surprises
17:00-18:00	Demos and judging	—

13 Pitch deck plan (for the PPTX)

Eight slides for a 2-minute pitch. One idea per slide, big visuals, few words. The live demo happens at slide 5.

#	Slide title	Key message / content	Visual
1	Stablecoin Circuit Breaker	Detect stress. Trigger the plan. Prove it on Solana.	Logo + three traffic lights
2	\$1 until it isn't	USDC fell to ~87¢ over a weekend in March 2023. Terra collapsed in 2022. Markets never close.	Simple price-dip line chart
3	Seeing it isn't the problem	Monitors already exist (show 3-4 logos/names). They stop at the alarm. No agreed response, no provable record.	Row of existing tools, with "alarm only" tags
4	Detect → Respond → Prove	Traffic-light rating; pre-approved policy with human sign-off; tamper-proof log on Solana.	Three-step diagram
5	Live demo	Play scenario C: amber → red → approve in Phantom → tokens move → Explorer.	Switch to the app
6	Why Solana	Notary (memo), sign-off (wallet signature), executor (token transfer): fast and near-free.	Explorer screenshot of a memo
7	Who needs this	Treasury, fintech and compliance teams; EU MiCA raises the bar on evidencing risk management.	User personas / MiCA badge
8	Honest limits and next steps	Flags, doesn't predict. Simulated data, devnet. Next: on-chain policy program, live feeds, calibrated thresholds.	Roadmap strip

2-minute script

Time	Say / show
0:00-0:20	"In March 2023, USDC lost 13% over a weekend. Everyone could see the price. What most firms lacked was an agreed plan, and proof of what they did." (slides 1-2)
0:20-0:40	"Depeg monitors already exist; here are a few. They stop at the alarm. We cover what happens next." (slides 3-4)
0:40-1:20	Live demo: play scenario C at 60x. Amber, red, approval card, sign in Phantom, balances change, open Explorer. (slide 5)
1:20-1:40	"Solana is our notary, our sign-off and our executor. Treasury and compliance teams need decisions in minutes and evidence for auditors." (slides 6-7)
1:40-2:00	"It flags stress; it doesn't predict. Thresholds are assumptions we'd calibrate. Next, we enforce the policy on-chain." (slide 8)

14 Risks, fallbacks and judge questions

Risk	Fallback
Solana install fails	Build and demo Tier 1 with no blockchain; show pre-recorded Explorer links.
Devnet transaction fails live	The app marks the event “not on-chain” and carries on; show the recording and README links.
Phantom approval is fiddly	Approve with an in-app button signed by the treasury key; present Phantom as the next step.
Tier 3 overruns	Hard stop at 15:00–15:30; the story still works with Tiers 1–2 and a mock-up of the approval card.
Claude usage runs low	Short, focused prompts; a fresh chat per tier; paste only the relevant file.

Likely judge questions

Question	Answer
How is this different from Webacy or DefiLlama?	They detect. We act on a pre-agreed policy, with a named approver, and prove what was done.
Can it predict depegs?	No. It flags stress early with rules anyone can check. Depegs are rare and often driven by confidence shocks.
Why does this need a blockchain?	A log the firm controls can be edited. A Solana record carries a public timestamp and the approver's signature that nobody can change afterwards.
Would a firm let software move its money?	Not automatically. A named human approves every move, following a policy the firm wrote itself.
Where do the thresholds come from?	Starting assumptions, slightly more cautious than the 0.5–1% bands common in existing tools; we would calibrate them on past events.
Is the data real?	The demo uses clearly labelled simulations, including a stylised March 2023 replay. Live Pyth prices are the stretch tier.

15 Scope, glossary and sources

In scope: 2–3 stablecoins, simulated scenarios, rule-based ratings, a 3-rule policy, devnet only.

Out of scope: predicting depegs, real money or trades, investment advice, a custom Rust program, real exchange swaps.

Term	Plain-English meaning
Stablecoin	A digital token designed to stay worth a fixed amount, usually \$1
Peg / depeg	The \$1 target, and the price sliding away from it
Treasury team	The people in a company who manage its cash
Policy	Rules agreed in advance for what to do when something goes wrong
Devnet	Solana's free test network; no real money
Test token (SPL)	A token you create on devnet to stand in for a real stablecoin
Memo program	A Solana feature for attaching a short text note to a transaction
Phantom	A wallet app used to hold Solana accounts and sign approvals
Solana Explorer	A public website where anyone can look up a Solana transaction
Oracle (Pyth)	A service that publishes real-world prices for use on a blockchain
MICA	EU regulation covering crypto assets, including stablecoins

Sources (market landscape, checked 25 September 2026)

- kcolbchain/depeg-monitor — github.com/kcolbchain/depeg-monitor
- CoinGecko, Stablecoin depeg detection guide — coingecko.com/learn/stablecoin-depeg-risk-detection
- Webacy Stablecoin Monitor — webacy.com/stablecoin-monitoring; docs.webacy.com
- S&P; Global Ratings, Stablecoin Stability Assessment launch (Dec 2023) — press.spglobal.com
- S&P; Global and Chainlink, assessments on-chain — nasdaq.com press release
- DefiLlama Stablecoins — defillama.com/stablecoins
- Apify Stablecoin Depeg Monitor — apify.com/goontilldawn/stablecoin-depeg-monitor

Solana commands and library names should be re-checked against current documentation on Friday, since tools change.